```
        public string Name { get; set; }                    ❼
    }
}
```

❶ The list of strings serves as the application model in this example.

❷ Web APIs use attribute routing to define the route templates.

❸ The [FromBody] attribute indicates that the parameter should be bound to the request body.

❹ You need to check if model validation succeeded and return a 400 response if it failed.

❺ If the data sent does not contain a valid ID, returns a 404 ProblemDetails response

❻ Updates the model and returns a 200 Response

❼ UpdateModel is valid only if the Name value is provided, as set by the [Required] attribute.

This example demonstrates many common features and patterns used with web API controllers:

- Web API controllers read data from the body of a request, typically sent as JSON. To ensure the body is read as JSON and not as form values, you have to apply the `[FromBody]` attribute to the method parameters to ensure it is model-bound correctly.
- As discussed in chapter 16, after model binding, the model is validated, but it's up to you to act on the validation results. You should return a `400 Bad Request` response if the values provided failed validation. You typically want to provide details of *why* the request was invalid: this is done in listing 20.10 by returning a `ValidationProblemDetails` object in the response body, built from the `ModelState`.
- Whenever you return an error status, such as a `404 Not Found`, where possible you should return details of the problem that will allow the caller to diagnose the issue. The `ProblemDetails` class is the recommended way of doing that in ASP.NET Core.